Studio Nx

AI 구현 설명 74 — S1-07 샘플 라이브러리

2026-10-16

AI 구현 설명 74 — S1-07 샘플 라이브러리 (업로드, 파형 데이터, 태그, 저장 용량)

항목	내용
작성	Claude (AI), 2026-10-15, 사용자 검토
작업	S1-07 샘플 라이브러리 (YouTrack StudioNx-38), PRD 02 FR-221, FR-222, FR-223, FR-224, FR-225, FR-226, FR-227
브랜치	`feature/phase1/s1-07-sample-library` (`develop`에서 만듦)
바꾼 파일	`backend/app/models/sample.py`, `models/tag.py`, `services/sample_audio_service.py`, `services/sample_service.py`, `services/tag_service.py`, `api/v1/samples.py`, `api/v1/tags.py`, `api/v1/router.py`, `schemas/sample.py`, `schemas/tag.py`, `core/config.py`, `core/errors.py`, `.env.example`, `requirements.txt`
진행 방식	**함수 하나씩** 구현 → 사용자 검토 → 다음. 실제 코드는 **전통 문법**(`?.`, `??` 없음), 최신 문법 버전은 함수 문서의 "최신 문법으로 쓰면" 절에
함수별 설명	`docs/ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/177 ~ 192` (md, PDF)
이전 문서	68_s1-06_timeline.md

1. 실행 결과

샘플을 올리고, 그 목록을 보고, 소리를 듣고, 파형을 그리고, 이름과 태그를 고치고, 지울 수 있는 **백엔드 API 한 벌**을 만들었습니다. 파일은 디스크에 두고(`samples/{owner_id}/{무작위}.{확장자}`), DB(`samples` 행)에는 그 정보와 파형 파일 경로를 담습니다. 올릴 때 길이·샘플레이트·채널을 읽어 채우고(FR-223), 파형 데이터를 JSON 파일로 만들어 둡니다.

화면 쪽(라이브러리 패널, 드래그앤드롭 업로드)은 **뒤 작업**입니다. 이번 작업은 그 화면이 부를 API까지입니다. 그래서 실행 화면 캡처는 없고, 시험은 실제 MySQL과 HTTP로 확인했습니다.

2. 한 일

S1-06까지 만든 시퀀서 화면에 소리를 채우려면 샘플이 필요합니다. 그 샘플을 다루는 부분을 맡았습니다.

파일	역할
`models/sample.py`	`Sample`(올린 파일 한 건), `SampleSource`(생긴 경로: UPLOAD 등)
`models/tag.py`	`Tag`, `TagCategory`(GENRE · INSTRUMENT · MOOD · ETC), `SampleTag`(샘플과 태그 연결)
`services/sample_audio_service.py`	오디오를 읽고 파형을 만드는 순수 계산: `read_audio_info`(180), `compute_waveform`(181), `write_waveform`(182), `check_upload`(183)
`services/sample_service.py`	저장·조회: `save_sample`(184), `list_samples`(186), `get_sample`, `sample_file`(187), `sample_waveform`(188), `update_sample`(189), `set_sample_tags`(190), `delete_sample`(191)
`services/tag_service.py`	태그 목록: `list_tags`(192)
`api/v1/samples.py`	샘플 라우터: 올리기(185), 목록(186), 파일(187), 파형(188), 이름(189), 태그(190), 삭제(191)
`api/v1/tags.py`	태그 라우터: 목록(192)
`schemas/sample.py`, `schemas/tag.py`	요청·응답 모양 (한 줄 `SampleResponse`, `TagItem` 등)

파일	역할
`core/config.py`, `core/errors.py`, `.env.example`	파일 크기 한도(`sample_max_bytes`, 기본 100MB), 오류 코드 `STORAGE_QUOTA_EXCEEDED`
`requirements.txt`	`soundfile` 0.14.0, `numpy` 2.5.4 (오디오 읽기·파형 계산)

3. 함수 16개 (번호는 P1-28의 176에 이어서)

#	함수	하는 일	설명
177	`Sample`, `SampleSource`	올린 파일 한 건의 정보를 담는 표와 생긴 경로	177
178	`Tag`, `TagCategory`	태그 표와 분류	178
179	`SampleTag`	샘플과 태그를 잇는 표 (다대다)	179
180	`read_audio_info`	파일 머리말에서 길이·샘플레이트·채널을 읽습니다	180
181	`compute_waveform`	소리를 1000칸으로 접어 `[최솟값, 최댓값]` 쌍을 만듭니다	181
182	`write_waveform`	파형을 JSON 파일로 저장합니다	182
183	`check_upload`	형식(415), 크기(413), 남은 용량(413)을 검사합니다 (FR-222·227)	183
184	`save_sample`	파일을 1MB씩 저장하고 검사·메타·파형·행·사용량을 한 트랜잭션으로 처리	184
185	`POST /samples`	샘플 올리기 (multipart)	185
186	`GET /samples`, `list_samples`	목록, 이름 검색, 태그 필터, 페이지 (FR-224)	186
187	`GET /samples/{id}/file`	오디오 파일 스트리밍, ETag·304·Range (FR-224 미리듣기)	187
188	`GET /samples/{id}/waveform`	파형 JSON 서빙 (FR-223·210)	188
189	`PATCH /samples/{id}`, `update_sample`	이름 변경 (FR-225)	189
190	`PUT /samples/{id}/tags`, `set_sample_tags`	태그를 통째로 교체 (FR-225)	190
191	`DELETE /samples/{id}`, `delete_sample`	소프트 삭제와 저장 용량 반환 (FR-225·135)	191
192	`GET /tags`, `list_tags`	태그 목록 (분류 · 이름 순)	192

4. 요구사항 대응

요구사항	내용	어디서
FR-221	파일을 화면에 드롭하면 업로드해 `samples`에 저장	저장은 184, 라우터는 185. 화면 드롭은 뒤 작업
FR-222	지원 형식 WAV·MP3·FLAC·OGG·AIFF, 파일당 100MB	183 `check_upload`, `SUPPORTED_AUDIO_EXTENSIONS`
FR-223	올릴 때 길이·샘플레이트·채널 추출, 파형 데이터 생성	180 읽기, 181 계산, 182 저장, 188 서빙
FR-224	목록, 이름 검색, 태그 필터, 클릭 시 미리듣기	186 목록·검색·필터, 187 미리듣기, 192 태그 목록
FR-225	이름 변경, 태그 추가·삭제, 소프트 삭제	189 이름, 190 태그, 191 삭제
FR-226	여러 파일 동시 드롭, 업로드 진행률	지금은 한 번에 한 파일. 뒤 작업으로 미룸
FR-227	남은 용량 확인, 초과 시 거부와 정리 안내	183 `STORAGE_QUOTA_EXCEEDED`, 184 사용량 증가, 191 반환

5. 처음 설계에서 달라진 점

항목	처음 계획	구현	이유
파형 데이터 모양	정해지지 않음	JSON 피크 배열(`[최솟값, 최댓값]` 쌍, 최대 1000칸)	서버가 그림을 만들지 않고 브라우저가 캔버스에 그립니다. 확대·색 바꾸기가 자유롭고 서버가 가볍습니다 (사용자 결정 2026-10-15)
형식 검사 실패	메타 칸을 비워 둠	415로 거부	`duration_ms`·`sample_rate`·`channels`가 NULL이 되지 않게 했습니다. 못 읽는 파일은 받지 않습니다
파일 저장과 사용량	따로 처리	한 트랜잭션	행·파일·사용량이 어긋나지 않게 한 커밋으로 묶었습니다 (FR-135). 반쪽 상태가 남지 않습니다
용량 초과 오류 코드	`FILE_TOO_LARGE` 하나	`STORAGE_QUOTA_EXCEEDED` 신설	파일 한도 초과(413)와 남은 용량 부족(413)은 안내가 달라야 합니다. 정리 안내를 보여 주려고 나눴습니다
태그 필터 기준	이름	`tag_id`	같은 이름이 분류별로 있을 수 있어 이름 대신 번호로 거릅니다. 화면도 `GET /tags`(192)에서 번호를 받아 씁니다
파일 열람 권한	본인과 프로젝트 멤버	지금은 본인만	그 샘플을 쓰는 클립이 아직 없습니다. 멤버 열람은 클립을 만들 때 함께 넣습니다
`mime_type` 결정	브라우저가 보낸 값	확장자 표로 결정	브라우저 `content_type`은 믿지 않습니다. 검사를 통과한 확장자로 정합니다

6. 시험

방법	대상	결과
`pytest` (실제 MySQL, 끝나면 rollback)	전체	**986개 통과**
`ruff check`	백엔드 전체	통과
함수 시험	177 ~ 192	모델 관계, 오디오 읽기·파형 계산, 저장·검증·사용량, 라우터 (401·404·413·415·422·304·206·204)

시험 파일	개수	확인한 것
`test_sample_model.py`, `test_tag_model.py`	5 + 4	표·관계·기본값·DB 일치
`test_sample_audio_service.py`	46	형식·크기·용량 판정, 파형 계산, JSON 저장
`test_sample_service.py`	12	저장(분할 쓰기·검증·사용량), 실패 시 파일 정리
`test_sample_routes.py`	53	업로드·목록·파일·파형·이름·태그·삭제의 모든 응답
`test_tag_routes.py`	4	태그 목록 401, 정렬

7. 남은 일과 다음 작업이 쓰는 곳

작업	쓰는 것
S1-07 화면 (라이브러리 패널, 드래그앤드롭 업로드)	이번 API를 부릅니다. HTML/CSS는 `frontend-reference/`를 보고, JS는 함수 하나씩
FR-226 다중 파일 업로드, 진행률	지금은 한 번에 한 파일. 응답을 배열로 바꾸거나 반복 호출로 처리
타임라인 파형 연결	S1-06의 `placeholderPeaks`(148) 자리에 188의 peaks를 넘깁니다
S1-10 저장·불러오기(`/full`)	프로젝트의 클립이 `samples`를 가리킵니다. 샘플 정보를 이 표에서 읽습니다

작업	쓰는 것
프로젝트 멤버 열람	파일·파형 서빙(187·188)에 클립 기준 멤버 확인을 더합니다
FR-137 정리 작업	소프트 삭제된 샘플의 오디오·파형 파일을 실제로 지웁니다
태그 만들기·고치기	화면 기획이 정해지면 `GET /tags`(192)에 이어 추가
이 작업에서 미룬 것	녹음(`source='RECORDING'`), 응답의 파일·파형 URL 칸, 다중 업로드는 뒤 작업입니다